

PageRank en Spark

Aurélien Duvignac-Rosa, Jean-Marc Fauvel, Edoardo Piciucchi

Résumé—Ce travail présente la mise en œuvre et l'évaluation de l'algorithme PageRank au sein de l'écosystème Apache Spark, selon deux approches distinctes : RDD, DataFrame, avec et sans partitionnement explicite. Les expérimentations ont été menées sur cinq graphes de tailles croissantes, en utilisant une configuration Spark systématique et détaillée dans ce rapport, afin de garantir la comparabilité et la stabilité des mesures.

Les résultats mettent en évidence une amélioration notable des performances pour la version RDD partitionnée par rapport à l'approche RDD classique, notamment sur les graphes de grande taille. Cependant, cette optimisation reste globalement moins efficace que les implémentations fondées sur les DataFrames, qui bénéficient directement des mécanismes d'optimisation du moteur Catalyst. De manière surprenante, le partitionnement manuel appliqué aux DataFrames n'apporte pas de gain significatif et tend même à dégrader légèrement les performances, en raison de la perte des optimisations automatiques de Spark SQL.

Ces observations confirment la supériorité des DataFrames pour le traitement distribué de graphes volumineux et soulignent l'importance d'adapter la stratégie de partitionnement à la structure de données utilisée.

Keywords—PageRank ; Spark ; Resilient Distributed Dataset ; DataFrame ; Catalyst ; Scala ; Garbage collector

I. PRÉSENTATION DU PROJET

Le projet consiste à implémenter l'algorithme de PageRank en utilisant deux approches distinctes : RDD et DataFrame. Les graphes utilisés proviennent de Wikipédia¹ (un graphe par langue), fournis au format texte, chaque ligne correspondant à une page suivie de ses liens sortants (cf. représentation ci-dessous).

```
page1 | lien1-1 | lien1-2 | ...
page2 | lien2-1 | lien2-2 | ...
```

Chaque page est ainsi associée à une liste de liens sortants. Certains de ces liens peuvent pointer vers des pages inexistantes (liens rouges de Wikipédia), qu'il faudra ignorer lors de la construction du graphe.

A. Objectifs

- Développer une première version *baseline* en Spark RDD, sans optimisation particulière (pas de partitionnement avancé) ;
- Développer ensuite une version optimisée afin d'améliorer l'efficacité de l'algorithme ;
- Réaliser une comparaison des performances entre l'approche RDD et DataFrame.

1. Accès via Dropbox : Jeu de graphes

B. Analyse expérimentale

L'expérimentation portera sur plusieurs graphes de tailles différentes, afin d'évaluer :

- la précision et la robustesse des résultats obtenus ;
- l'impact de la taille du graphe sur le temps d'exécution ;
- les différences de performance entre les deux approches Spark.

L'ensemble des expérimentations présentées ci-après a été réalisé en Scala, en utilisant les API RDD et DataFrame de Spark.

C. Rendu souhaité

Le rapport doit présenter la méthodologie, les implémentations, ainsi que l'analyse comparative des résultats obtenus.

II. INTRODUCTION

Aux débuts du web, les moteurs de recherche reposaient sur des approches basées principalement sur le contenu textuel des pages, en indexant les mots-clefs extraits par les robots d'exploration. Ces méthodes, bien que simples à mettre en œuvre, se révélaient facilement manipulables et peu fiables. Divers indicateurs, tels que les appréciations des utilisateurs ou l'analyse des clics, furent également envisagés sans réellement résoudre ces limites.

L'émergence d'une approche structurale a marqué un tournant majeur : il a été reconnu que le web forme un réseau d'hyperliens dont certaines pages occupent une position centrale. L'importance d'une page ne dépend donc pas uniquement de son contenu, mais aussi du nombre et de la qualité des liens qui y conduisent.

C'est sur ce principe qu'a été conçu l'algorithme PageRank, élaboré par Larry Page et Sergey Brin [1], alors étudiants en doctorat à l'Université de Stanford. S'en est suivi le dépôt d'un brevet en 1998 [2]. En modélisant le web comme un graphe orienté, PageRank attribue à chaque page une valeur reflétant son importance relative, offrant ainsi un classement plus pertinent et résistant aux manipulations que les approches antérieures.

III. ALGORITHME PAGERANK

Dans l'algorithme PageRank, la valeur attribuée à une page découle à la fois de sa « pertinence » et de son « importance » dans la structure du réseau :

- La « pertinence » dépend du contenu de la page : elle est jugée élevée lorsqu'elle contient des mots-clefs associés à la requête de l'utilisateur ;
- L'« importance » découle de la structure du web : une page est d'autant plus importante qu'elle reçoit de

nombreux liens entrants, notamment depuis d'autres pages elles-mêmes centrales, souvent situées au cœur des Sites fortement connectés (SFC) ;

Afin de formaliser cette intuition, le modèle propose de simuler le comportement d'un « surfeur aléatoire » parcourant le web. Celui-ci quitte spontanément les pages peu pertinentes ou isolées pour se diriger vers celles jugées plus centrales et connectées.

L'objectif consiste à déterminer la probabilité qu'un internaute se trouve sur une page donnée après un grand nombre d'étapes. Cette probabilité constitue le « score PageRank », qui mesure l'importance relative des pages au sein du graphe et dont le calcul est présenté dans l'algorithme 1.

Algorithme 1 Calcul du PageRank avec facteur d'amortissement

Entrées : Graphe orienté $G = (V, E)$, facteur d'amortissement $d \in [0, 1]$, seuil de convergence ε , nombre maximal d'itérations K

Sorties : Score $PR(v)$ pour chaque page $v \in V$

- 1: **Initialisation** : $PR(v) \leftarrow \frac{1}{|V|}$ pour tout $v \in V$
- 2: Calculer $\text{outdeg}(v)$, le nombre de liens sortants de chaque nœud
- 3: **Pour** $k = 1$ **to** K **faire**
- 4: $PR_{\text{new}}(v) \leftarrow \frac{1-d}{|V|}$ pour tout $v \in V$
- 5: **Pour** chaque arête $(u, v) \in E$ **faire**
- 6: $PR_{\text{new}}(v) \leftarrow PR_{\text{new}}(v) + d \times \frac{PR(u)}{\text{outdeg}(u)}$
- 7: **Fin Pour**
- 8: Redistribuer le score des nœuds *Dangling nodes* :

$$PR_{\text{new}}(v) \leftarrow PR_{\text{new}}(v) + d \times \frac{S}{|V|}, \quad S = \sum_{u \in D} PR(u)$$
- 9: **Si** $\sum_{v \in V} |PR_{\text{new}}(v) - PR(v)| < \varepsilon$ **alors**
- 10: **Arrêter la boucle**
- 11: **Fin Si**
- 12: $PR(v) \leftarrow PR_{\text{new}}(v)$ pour tout $v \in V$
- 13: **Fin Pour**
- 14: **Retourner** $PR(v)$ pour tout v

A. Formulation mathématique

Le comportement du surfeur aléatoire est modélisé par une chaîne de Markov. Le web est représenté par un graphe orienté où les nœuds correspondent aux pages et les arêtes aux hyperliens. À chaque arête est associée une probabilité de transition : la probabilité de passer d'une page à une autre.

La probabilité stationnaire $PR(i)$ d'être sur une page i satisfait la relation :

$$PR(i) = \frac{1-d}{N} + d \sum_{j \in M(i)} \frac{PR(j)}{L(j)} \quad (1)$$

où :

- N est le nombre total de pages ;

- $d \in [0, 1]$ est le Facteur d'amortissement, typiquement 0,85 ;
 - $M(i)$ est l'ensemble des pages pointant vers i ;
 - $L(j)$ est le nombre de liens sortants de la page j .
- En tenant compte des *Dangling nodes*, on obtient :

$$PR(i) = \frac{1-d}{N} + d \left(\sum_{j \in M(i)} \frac{PR(j)}{L(j)} + \frac{1}{N} \sum_{k \in D} PR(k) \right) \quad (2)$$

où D désigne l'ensemble des nœuds sans lien sortant.

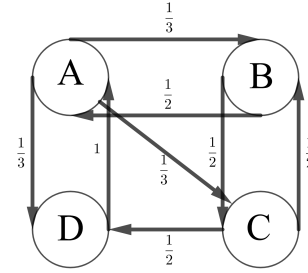


FIGURE 1. Exemple de graphe pour l'illustration du calcul du PageRank.

À partir du graphe de la figure 1, la Matrice de transition correspondante est donnée par :

$$M = \begin{bmatrix} 0 & \frac{1}{2} & 1 & 0 \\ \frac{1}{3} & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{2} & 0 & 0 \end{bmatrix}.$$

Chaque colonne de M représente les probabilités de transition à partir d'une page donnée, et la matrice est stochastique.

Si v_i désigne la distribution des probabilités après i itérations, alors :

$$v_i = M v_{i-1}, \quad v_i = M^i v_0,$$

où v_0 est la distribution initiale uniforme :

$$v_0 = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{bmatrix}.$$

B. Calcul matriciel des premières itérations

À partir de la matrice M et du vecteur initial v_0 , on obtient :

a) *Première itération*:

$$v_1 = M v_0 = \begin{bmatrix} \frac{3}{8} \\ \frac{5}{24} \\ \frac{5}{24} \\ \frac{5}{24} \end{bmatrix}.$$

b) *Itérations suivantes:*

$$v_2 = \begin{bmatrix} \frac{5}{16} \\ \frac{11}{48} \\ \frac{11}{48} \\ \frac{11}{48} \end{bmatrix}, \quad v_3 = \begin{bmatrix} \frac{11}{32} \\ \frac{7}{32} \\ \frac{7}{32} \\ \frac{7}{32} \end{bmatrix}.$$

Après un nombre suffisant d'itérations, la suite $\{v_i\}$ converge vers la distribution stationnaire :

$$v_{50} = \begin{bmatrix} \frac{1}{3} \\ \frac{2}{9} \\ \frac{2}{9} \\ \frac{2}{9} \end{bmatrix}.$$

Cette distribution traduit la probabilité limite d'occurrence sur chaque page : le nœud *A* concentre $\frac{1}{3}$ de la probabilité totale, tandis que *B*, *C* et *D* partagent chacun $\frac{2}{9}$.

C. Exemple d'une première itération du PageRank

Le graphe de la figure 1 peut être décrit par :

$$\begin{cases} A \rightarrow \{B, C, D\} \\ B \rightarrow \{A, C\} \\ C \rightarrow \{B, D\} \\ D \rightarrow \{A\} \end{cases}$$

a) *Initialisation:*

$$v_{(0)} = \begin{bmatrix} 0.25 & 0.25 & 0.25 & 0.25 \end{bmatrix}^T.$$

b) *Distribution des contributions:*

$$\begin{aligned} A : 0.25/3 &= 0.0833 \quad \text{vers } B, C, D, \\ B : 0.25/2 &= 0.125 \quad \text{vers } A, C, \\ C : 0.25/2 &= 0.125 \quad \text{vers } B, D, \\ D : 0.25/1 &= 0.25 \quad \text{vers } A. \end{aligned}$$

c) *Somme des contributions reçues:*

$$\begin{aligned} A &= 0.125 + 0.250 = 0.375, \\ B &= 0.0833 + 0.125 = 0.2083, \\ C &= 0.0833 + 0.125 = 0.2083, \\ D &= 0.0833 + 0.125 = 0.2083. \end{aligned}$$

d) *Vecteur après une itération:*

$$v_{(1)} = \begin{bmatrix} 0.375 & 0.2083 & 0.2083 & 0.2083 \end{bmatrix}^T$$

L'algorithme PageRank garantit la conservation de la masse de probabilité, la somme des composantes du vecteur restant égale à 1. En effet : $\sum_i v_i^{(1)} = 1$.

Dans l'exemple présenté, le vecteur de probabilité stationnaire obtenu au bout de 50 itérations est :

$$v_{(50)} = \begin{bmatrix} \frac{1}{3} & \frac{2}{9} & \frac{2}{9} & \frac{2}{9} \end{bmatrix}^T$$

Soit numériquement :

$$v_{(50)} = \begin{bmatrix} 0.3333 & 0.2222 & 0.2222 & 0.2222 \end{bmatrix}^T$$

La figure 2 illustre l'évolution du vecteur de probabilité associé au graphe de la figure 1. Le calcul a été effectué avec un facteur d'amortissement ($d = 1$), afin d'observer la convergence naturelle du processus de Markov sans composante de téléportation. L'obtention de ce résultat s'est fait à l'aide de l'implémentation RDD sans partitionnement, décrite dans la section IV-C.

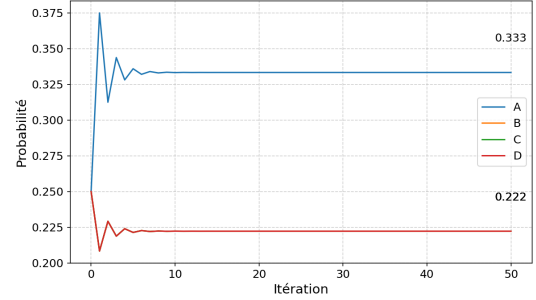


FIGURE 2. Convergence du graphe de la figure 1 sur 50 itérations, obtenue à l'aide de l'implémentation RDD sans partitionnement de l'algorithme PageRank.

Cette étude de convergence met en évidence le comportement stationnaire du modèle PageRank.

Abordons à présent l'implémentation en Spark, en détaillant les approches RDD et DataFrame ainsi que les optimisations adoptées pour le traitement distribué.

IV. IMPLÉMENTATION SCALA DU PAGERANK

L'implémentation de PageRank a été réalisée en Scala au sein de l'environnement Apache Spark². Ce choix repose sur la capacité de Spark à distribuer efficacement les calculs sur de larges volumes de données tout en offrant des abstractions adaptées à la manipulation de graphes. L'objectif de cette section est de détailler les différentes étapes de cette implémentation, depuis la structure générale du projet jusqu'à l'étude comparative des versions RDD et DataFrame, avec et sans partitionnement explicite.

A. Différences entre RDD et DataFrame

Spark propose deux modèles de calcul principaux : les Resilient Distributed Dataset (RDD) et les DataFrames. Les RDDs constituent l'abstraction fondamentale de Spark, représentant un ensemble distribué d'éléments partitionnés entre les nœuds du Cluster. Ils offrent un contrôle fin sur les opérations et le partitionnement, mais nécessitent parfois une gestion explicite des transformations et des types de données.

2. L'ensemble de l'implémentation est disponible sur ce lien github : <https://github.com/auduvignac/pagerank-spark>

Les DataFrames, quant à eux, reposent sur une abstraction de niveau supérieur inspirée du modèle relationnel. Ils permettent d'exprimer les transformations sous forme déclarative, optimisée par le moteur Catalyst. Bien que plus efficaces sur de nombreux cas d'usage, ils introduisent une couche d'optimisation automatique qui peut masquer certains détails de l'exécution.

Ainsi, les expérimentations présentées dans ce travail comparent les performances et comportements des deux approches, dans un contexte identique de calcul du PageRank.

B. Structure du projet

Le projet a été structuré de manière modulaire afin de faciliter la comparaison entre les différentes variantes d'implémentation. L'organisation générale se compose des éléments suivants :

- GraphRDD et GraphDF : classes de représentation du graphe en RDD et en DataFrame ;
- PageRankRDD et PageRankDF : implémentations respectives de l'algorithme PageRank ;
- Main : point d'entrée du programme, permettant de sélectionner le mode d'exécution, le nombre de partitions et le fichier d'entrée ;
- Benchmark : module dédié à la collecte des métriques et à la comparaison des performances.

Pour garantir la cohérence des résultats sur les différents graphes, des tests unitaires ont été mis en place³ pour s'assurer du bon fonctionnement des classes utilisées et du bon fonctionnement de l'algorithme.

Pour chaque structure de données, l'implémentation de l'algorithme PageRank s'articule autour d'une phase d'initialisation suivie d'une boucle itérative de calcul.

C. Implémentation RDD sans partitionnement

Cette première version s'appuie sur les mécanismes de partitionnement par défaut de Spark. Le graphe est chargé sous la forme d'un RDD de paires (page, liens_sortants), sur lequel sont appliquées les transformations définies par l'algorithme PageRank. Bien que simple à mettre en œuvre, cette approche ne tire pas pleinement parti de la localité des données, ce qui peut engendrer des coûts de communication importants lors des opérations de join et d'aggrégation.

a) *Initialisation*: En amont de la phase itérative, les structures nécessaires au calcul du PageRank sont initialisées. Le nombre total de pages N est d'abord déterminé à partir de l'attribut `nNodes` de l'instance du graphe créé, puis les ensembles des nœuds et des liens sont chargés en mémoire distribuée et rendus persistants afin d'éviter leur rechargement à chaque itération.

```
val N: Double = graph.nNodes.toDouble
val allNodes: RDD[String] = graph.allNodes.persist(storage)
val links: RDD[(String, Seq[String])] = graph.links
```

3. <https://github.com/auduvignac/pagerank-spark/tree/main/src/test/scala/pagerank>

Chaque nœud est ensuite associé à la liste de ses liens sortants ainsi qu'à son degré sortant $L(j)$. Les doublons sont supprimés afin de garantir une représentation unique des arêtes.

```
val outMap: RDD[(String, (Int, Seq[String]))] = links
  .reduceByKey(_ ++ _)
  .mapValues(_.distinct)
  .mapValues(outs => (outs.size, outs))
```

Les nœuds dépourvus de liens sortants (*Dangling nodes*) sont ajoutés à la structure, avec un degré nul et une liste vide, afin de conserver un graphe complet et cohérent.

```
val nodesWithOut: RDD[(String, (Int, Seq[String]))] =
  allNodes
  .map(nodeId => (nodeId, (0, Seq.empty[String])))
  .leftOuterJoin(outMap)
  .mapValues {
    case ((_, _), Some((deg, outs))) => (deg, outs)
    case _ => (0, Seq.empty[String])
  }
  .persist(storage)
```

Enfin, le vecteur des rangs v_0 est initialisé de manière uniforme :

$$v_0 = \left[\frac{1}{N}, \frac{1}{N}, \dots, \frac{1}{N} \right]^T$$

ce qui correspond à une probabilité égale pour chaque page d'être visitée au départ.

```
var ranks: RDD[(String, Double)] = allNodes
  .map(n => (n, 1.0 / N))
  .persist(storage)
```

b) *Boucle itérative (fonction oneStep)* :: La fonction `oneStep` réalise une itération complète du calcul du PageRank, selon le principe de mise à jour :

$$v_{i+1} = Mv_i.$$

Elle est appelée successivement jusqu'à atteindre le nombre maximal d'itérations défini dans la configuration du benchmark⁴.

```
def oneStep(
  ranks: RDD[(String, Double)],
  nodesWithOut: RDD[(String, (Int, Seq[String]))],
  allNodes: RDD[String],
  N: Double,
  damping: Double = 1.0,
  debug: Boolean = false,
  logger: Logger
): RDD[(String, Double)] = {
```

Calcul de la masse des *Dangling nodes* Les pages sans lien sortant transfèrent leur poids total à la masse globale `danglingMass`, qui sera redistribuée uniformément sur l'ensemble du graphe.

```
val danglingMass = nodesWithOut.join(ranks)
  .filter { case (_, ((deg, _), _)) => deg == 0 }
  .map { case (_, ((_, _), r)) => r }
  .sum()
```

4. Certaines implémentations interrompent le calcul dès que la convergence est atteinte. Dans le présent travail, l'objectif a été de se concentrer exclusivement sur les temps de traitement générés par les approches RDD et DataFrame, indépendamment du critère de convergence.

Calcul des contributions des nœuds *non-dangling*

Les pages disposant de liens sortants distribuent leur score proportionnellement à leur degré sortant. Chaque destination d reçoit une contribution $PR(j)/L(j)$ provenant de la page j .

```
val contribs = nodesWithOut.join(ranks)
  .flatMap { case (_, ((deg, outs), r)) =>
    if (deg == 0) Iterator.empty
    else outs.iterator.map(d => (d, r / deg))
  }
  .reduceByKey(_ + _)
```

Calcul du terme uniforme et redistribution de la masse des *Dangling nodes* Le facteur d'amortissement d (ou *damping factor*) est appliqué afin de simuler le comportement d'un internaute aléatoire, conformément au modèle de navigation décrit en section 2.

```
val base = ((1.0 - damping) / N) + (damping * danglingMass / N)
```

Calcul des nouveaux rangs Chaque page reçoit à la fois le terme uniforme et les contributions des pages qui pointent vers elle. L'opération `leftOuterJoin` garantit l'inclusion de toutes les pages, y compris celles sans lien entrant.

```
val newRanks = allNodes
  .map(id => (id, base))
  .leftOuterJoin(contribs)
  .mapValues { case (b, cOpt) => b + damping * cOpt.
    getOrElse(0.0) }
```

Normalisation du vecteur de rangs La somme totale des rangs est recalculée afin de garantir la propriété de normalisation :

$$\sum_i PR(i) = 1.$$

Cette étape assure que le vecteur de rangs demeure une distribution de probabilité valide.

```
val sumRanks = newRanks.values.sum()
val normalizedRanks = newRanks.mapValues(_ / sumRanks)
normalizedRanks
```

c) Analyse: Cette première implémentation met directement en œuvre la formulation mathématique du PageRank. Elle présente l'avantage de la simplicité et d'une lecture claire du flux de calcul distribué. Cependant, en l'absence de partitionnement explicite, les transformations comme `join` ou `reduceByKey` entraînent des échanges de données fréquents entre les nœuds du Cluster, ce qui limite la scalabilité sur des graphes de grande taille.

Dans la section suivante, une version optimisée introduit un partitionnement cohérent des RDDs, afin de réduire les coûts de communication et d'améliorer la localité des calculs.

D. Implémentation RDD avec partitionnement

Cette seconde version améliore la précédente en introduisant un partitionnement explicite des RDDs. L'objectif est de minimiser les transferts de données entre les nœuds du Cluster lors des opérations de `join` et de `reduceByKey`, qui constituent les étapes les plus coûteuses du calcul du PageRank.

a) Principe général: Alors que la version précédente reposait sur le partitionnement implicite de Spark, cette implémentation définit un partitionneur explicite de type `HashPartitioner`. Celui-ci garantit que les RDDs manipulés (`ranks`, `nodesWithOut`, `outMap`) sont répartis selon la même logique de hachage des clefs, ce qui permet aux jointures successives de s'exécuter localement, sans provoquer de *shuffle*.

```
val P = new HashPartitioner(
  if (numParts > 0) numParts
  else sc.defaultParallelism * 2
)
```

Le « partitionneur » est ensuite appliqué systématiquement à toutes les structures manipulées :

- `outMap` et `nodesWithOut` sont construits avec le « partitionneur » P ;
- le vecteur `ranks` est également partitionné selon P ;
- toutes les jointures ultérieures (`join`, `leftOuterJoin`) bénéficient ainsi d'une co-localisation des données.

Cette homogénéité de partitionnement permet d'éviter le déclenchement automatique de *shuffle*, l'une des principales sources de surcharge lors des traitements distribués sur des graphes volumineux.

b) Différences principales avec la version sans partitionnement: L'algorithme implémenté dans la méthode `oneStep` conserve la même logique computationnelle que celle décrite dans la section précédente : calcul de la masse des *Dangling nodes*, distribution des contributions, application du facteur d'amortissement et normalisation finale. Cependant, plusieurs optimisations structurelles sont introduites :

- 1) Jointure unique et persistée :** Les informations des nœuds et des rangs sont réunies en une seule jointure stockée en mémoire :

```
val joined = nodesWithOut.join(ranks).persist(storage)
```

Cela permet de réutiliser la même structure à la fois pour le calcul de la masse des *Dangling nodes* et pour celui des contributions, réduisant ainsi le coût global de calcul.

- 2) Utilisation du « partitionneur » dans les agrégations :** Les réductions sont désormais exécutées avec le « partitionneur » P , ce qui garantit la localité des opérations :

```
.reduceByKey(partitioner, _ + _)
```

Cette optimisation élimine les transferts inter-partitions inutiles lors de l'agrégation des contributions.

- 3) Évaluation locale des sommes partielles :** La somme totale des rangs est calculée par partition avant réduction globale :

```
val sumRanks = newRanks.mapPartitions(it => Iterator(
  it.map(_._2).sum)).sum()
```

Ce procédé limite les échanges de données et exploite efficacement le parallélisme offert par les exécuteurs Spark.

- 4) **Structures compactes et typées** : Les liens sortants sont stockés sous forme de `Array[String]` plutôt que de `Seq[String]`, permettant un accès séquentiel plus rapide et une empreinte mémoire réduite sur le Cluster.

c) *Synthèse comparative*: Le tableau 1 résume les principales différences entre les deux implémentations.

TABLEAU 1
COMPARAISON ENTRE LES IMPLÉMENTATIONS RDD SANS ET AVEC PARTITIONNEMENT

Aspect	RDD sans partitionnement	RDD avec partitionnement
Partitionnement	Automatique (par Spark)	HashPartitioner explicite appliqué à tous les RDD
Jointures	Chaque join déclenche un <i>shuffle</i>	Jointures locales (même partitionneur)
Persistance	Limitée aux RDD principaux	Persistences intermédiaires pour éviter recomputation
Type de structure	<code>Seq[String]</code>	<code>Array[String]</code> (plus compact et rapide)
Calcul des sommes	Réduction globale (<i>sum</i>)	Réduction locale par partition (<i>mapPartitions</i>)
Coût réseau	Élevé (multiples transferts)	Réduit (localité des données)
Performance	Simplicité d'exécution, bonne lisibilité	Optimisée pour la scalabilité sur grands graphes

d) *Analyse*: L'ajout d'un « partitionneur » cohérent entre les RDDs constitue une optimisation majeure dans le calcul du PageRank. Il permet d'exploiter la localité des données, de réduire les coûts de communication entre les exécuteurs et d'améliorer la reproductibilité des performances sur des graphes massifs.

Cette approche se rapproche d'une configuration typique de production, où le partitionnement et la persistance des structures sont ajustés pour garantir un compromis optimal entre consommation mémoire et débit de traitement. La section suivante présente la transposition de cette logique à l'approche DataFrame, exploitant cette fois le moteur d'exécution Catalyst.

E. Implémentation DataFrame sans partitionnement

Cette version s'appuie sur les opérations de haut niveau du moteur Catalyst, qui optimise automatiquement les plans d'exécution grâce à un analyseur logique et physique. L'objectif est d'exploiter les capacités de projection, d'agrégation et de jointure de DataFrame tout en conservant la logique mathématique décrite précédemment.

a) *Initialisation des structures de données*: Les différentes étapes de préparation consistent à structurer le graphe sous forme de tables relationnelles distribuées, en calculant les degrés sortants et en identifiant les *Dangling nodes*.

```
val N: Double = graph.nNodes.toDouble

val edges = graph.links
  .filter(col("outlink").isNotNull)
  .select(col("page").as("src"), col("outlink").as("dest"))
  .distinct()
  .persist(storage)
```

La table `edges` représente l'ensemble des arêtes du graphe, c'est-à-dire les transitions (u, v) entre pages. Les nœuds sont ensuite extraits à partir des colonnes `src` et `dest` :

```
val nodes = edges.select(col("src").as("id"))
  .union(edges.select(col("dest").as("id")))
  .distinct()
  .persist(storage)
```

Le degré sortant de chaque page est obtenu par un `groupBy` et un `count` :

```
val outdeg = edges
  .groupBy(col("src"))
  .agg(count(lit(1)).as("outdeg"))
  .withColumnRenamed("src", "id")
  .persist(storage)
```

Les nœuds sont ensuite enrichis avec leur degré sortant, les nœuds absents de `outdeg` étant considérés comme *Dangling nodes* ($L(j) = 0$) :

```
val nodesWithDeg = nodes
  .join(outdeg, Seq("id"), "left")
  .na.fill(0, Seq("outdeg"))
  .persist(storage)
```

L'ensemble des pages sans lien sortant est ensuite identifié :

```
val danglingIds = nodesWithDeg
  .filter(col("outdeg") === 0)
  .select(col("id"))
  .persist(storage)
```

Enfin, le vecteur initial des rangs est uniformément réparti sur toutes les pages :

$$v_0 = \left[\frac{1}{N}, \frac{1}{N}, \dots, \frac{1}{N} \right]^T$$

```
var ranks = nodes
  .withColumn("rank", lit(1.0 / N))
  .persist(storage)
  .localCheckpoint(eager = true)
```

b) *Itération principale (fonction oneStep)*: La fonction `oneStep` reproduit la même logique mathématique que celle présentée pour les RDDs, mais implémentée ici à l'aide d'opérations SQL distribuées.

```
def oneStep(
  ranks: DataFrame,
  edges: DataFrame,
  nodes: DataFrame,
  outdegPos: DataFrame,
  danglingIds: DataFrame,
```



```
N: Double,
damping: Double = 1.0,
storage: StorageLevel = StorageLevel.MEMORY_ONLY,
debug: Boolean = false,
logger: Logger
)(implicit spark: SparkSession): DataFrame = {
```

Masse des Dangling nodes Les pages sans liens sortants sont identifiées et leur masse de probabilité totale est sommée à l'aide d'une jointure en mode `left_semi` :

```
val danglingMass: Double =
  ranks.join(danglingIds, Seq("id"), "left_semi")
    .select(col("rank"))
    .rdd.map(_.getDouble(0)).sum()
```

Cette étape correspond au calcul de $S = \sum_{u \in D} PR(u)$ dans la formule (2).

Calcul des contributions Chaque page *non-dangling* distribue sa probabilité à ses destinations. L'opération s'effectue via deux jointures successives : `edges ↔ ranks` et `edges ↔ outdegPos`, suivie d'une agrégation par `dest` :

```
val contribs = edges
  .join(
    ranks.select(col("id").as("src"), col("rank").as("rank_src")),
    Seq("src", "inner")
  )
  .join(outdegPos, Seq("src"), "inner")
  .withColumn("contrib", col("rank_src") / col("outdeg"))
  .groupBy(col("dest").as("id"))
  .agg(sum(col("contrib")).as("sumContrib"))
  .persist(storage)
```

Application du facteur d'amortissement Le facteur d est appliqué à la fois sur les contributions et sur le terme uniforme $\frac{1-d}{N}$, auquel s'ajoute la redistribution de la masse des *Dangling nodes* :

$$PR(i) = \frac{1-d}{N} + d \left(\sum_{j \in M(i)} \frac{PR(j)}{L(j)} + \frac{S}{N} \right).$$

```
val base = ((1.0 - damping) / N) + (damping * danglingMass / N)
```

Les nouveaux rangs sont calculés en effectuant une jointure externe entre les nœuds et leurs contributions :

```
val newRanks = nodes
  .join(contribs, Seq("id"), "left")
  .select(
    col("id"),
    (lit(base) + lit(damping) * coalesce(col("sumContrib"), lit(0.0))).as("rank")
  )
  .persist(storage)
```

Normalisation du vecteur de rangs Comme dans la version RDD, la somme totale des rangs est ramenée à 1 pour assurer la cohérence probabiliste.

```
val sumRanks = newRanks.select(col("rank")).rdd
  .mapPartitions(it =>
    Iterator(it.map(_.getDouble(0)).sum)
  ).sum()

val normalizedRanks = newRanks
  .withColumn("rank", col("rank") / lit(sumRanks))
  .localCheckpoint(eager = true)
```

c) *Analyse comparative*: L'approche `DataFrame` offre une abstraction plus déclarative que les RDDs. Elle repose sur des transformations logiques optimisées par le moteur Catalyst, qui choisit automatiquement les stratégies de jointure et les plans physiques les plus efficaces.

Cependant, en l'absence de partitionnement explicite, les jointures entre `edges`, `ranks` et `outdeg` entraînent des opérations de *shuffle* réseau analogues à celles observées dans la version RDD non partitionnée. Par ailleurs, cette approche permet une formulation plus compacte, compatible avec les API SQL et les extensions du moteur `DataFrame`.

La section suivante présente la version partitionnée, qui applique les principes de co-localisation des données déjà explorés dans le cas des RDDs.

F. Implémentation `DataFrame` avec partitionnement

Cette implémentation vise à améliorer la localité des opérations et à réduire les coûts de *shuffle* réseau, tout en bénéficiant des optimisations du moteur d'exécution Catalyst.

Contrairement à la version précédente, où le partitionnement était implicite, cette implémentation répartit explicitement les `DataFrames` selon des colonnes clefs (`src` ou `id`), afin de garantir la co-localisation des données lors des jointures successives.

a) *Initialisation et partitionnement*: Un nombre de partitions p est défini à partir du degré de parallélisme du Cluster :

```
val p = if (numParts > 0) numParts else spark.sparkContext.defaultParallelism * 2
```

Toutes les structures de base (`edges`, `nodes`, `outdeg`, `ranks`) sont ensuite explicitement réparties sur ces partitions.

- Les arêtes sont partitionnées par plage de valeurs de la colonne `src` :

```
.repartitionByRange(p, col("src"))
```

ce qui assure que toutes les arêtes partageant la même source sont regroupées sur la même partition.

- Les nœuds et le vecteur de rangs sont également répartis par `id`, garantissant un alignement parfait entre les structures :

```
.repartition(p, col("id"))
```

- Le degré sortant (`outdeg`) est calculé localement sur chaque partition, puis répartitionné sur la clef `id` afin de limiter les échanges de données entre partitions.

Ainsi, les jointures successives entre `edges`, `ranks` et `outdeg` peuvent être effectuées localement sans déclencher de *shuffle* global.

b) *Itération principale (fonction `oneStep`)* :: La logique de calcul reste identique à celle de la version non partitionnée : calcul de la masse des *Dangling nodes*, distribution des contributions, application du facteur d'amortissement et normalisation du vecteur des rangs.

Cependant, plusieurs différences notables ont pour vocation à renforcer l'efficacité de cette implémentation :

- 1) **Optimisation du calcul de la masse des *Dangling nodes*** Le calcul de la masse totale est effectué via l'API RDD, afin d'éviter la matérialisation complète du plan logique Catalyst :

```
val danglingMass: Double =
  ranks.join(danglingIds, Seq("id"), "left_semi")
    .select(col("rank"))
    .rdd.map(_._2.getDouble(0)).sum()
```

Cette approche combine la simplicité du modèle DataFrame avec la performance du modèle bas-niveau `RDD.sum()`.

- 2) **Jointures locales et partitionnées** Toutes les jointures sont réalisées sur des colonnes déjà alignées et partitionnées, évitant ainsi la redistribution des données :

```
.join(outdegPos, Seq("src"), "inner")
```

Contrairement à la version non partitionnée, aucune réorganisation globale du plan n'est nécessaire.

- 3) **Calcul distribué des contributions** Les contributions sont regroupées par clef locale grâce à `groupBy(col("dest"))`, puis agrégées par somme. Chaque partition traite ainsi son sous-ensemble du graphe en quasi-autonomie, réduisant la charge sur le moteur d'exécution Catalyst.

- 4) **Normalisation locale** La somme totale des rangs est obtenue via un calcul distribué par partition :

```
val sumRanks = newRanks.select(col("rank")).rdd
  .mapPartitions(it => Iterator(it.map(_._2.getDouble(0))
    .sum()))
  .sum()
```

Cette technique réduit considérablement le volume des données agrégées globalement.

c) *Analyse comparative*: Le tableau 2 résume les différences principales entre les versions DataFrame sans et avec partitionnement.

TABLEAU 2
COMPARAISON ENTRE LES IMPLÉMENTATIONS DATAFRAME SANS ET AVEC PARTITIONNEMENT

Aspect	DF sans partitionnement	DF avec partitionnement
Partitionnement	Automatique (Catalyst)	Explicite via <code>repartitionByRange</code> ou <code>repartition(p, col("id"))</code>
Jointures	Multiples <i>shuffle</i> réseau	Jointures locales entre partitions alignées
Plan d'exécution	Recalcul complet à chaque itération	Réutilisation locale des partitions et persistance
Calcul des sommes	Global via Catalyst	Local via <code>RDD.mapPartitions</code>
Coût réseau	Élevé (redistribution implicite)	Réduit (co-localisation explicite des données)
Performance	Dépendante du plan Catalyst	Optimisée pour la scalabilité et la localité
Lisibilité du code	Plus concise et déclarative	Plus technique, mais mieux contrôlée

d) *Synthèse*: Le passage d'une approche DataFrame non partitionnée à une version partitionnée permet d'exploiter plus efficacement la parallélisation intra-cluster. En combinant la co-localisation des données avec les optimisations automatiques du moteur Catalyst, cette version constitue un compromis performant entre le modèle déclaratif de Spark SQL et la maîtrise fine du traitement distribué.

Elle se rapproche ainsi du comportement de la version RDD partitionnée tout en conservant les avantages analytiques et expressifs du modèle DataFrame.

Les quatre implémentations présentées traduisent des choix de modélisation et d'optimisation distincts du calcul du PageRank. Chacune d'elle met en évidence un compromis entre simplicité de mise en œuvre, maîtrise du partitionnement, et efficacité d'exécution au sein du Cluster.

Afin d'évaluer de manière objective leurs performances respectives, une succession de tests a été réalisée sur différents graphes représentatifs. L'objectif est d'analyser l'impact du modèle de données et des stratégies de partitionnement sur les temps d'exécution, la charge du *garbage collector*, et la consommation des ressources mémoire.

Pour assurer une comparaison équitable entre implémentation et taille de graphe, la configuration Spark utilisée est la même pour chaque exécution.

V. CONFIGURATION SPARK UTILISÉE POUR LES MESURES

L'ensemble des expérimentations a été réalisé à l'aide de la configuration suivante, appliquée lors de l'exécution du programme via la commande `spark-submit` sur le Cluster Lamsade :

```
spark-submit \
--class pagerank.Main \
--deploy-mode client \
--executor-cores 4 \
--num-executors 8 \
--executor-memory 10G \
--driver-memory 6G \
--conf "spark.serializer=org.apache.spark.serializer.
    KryoSerializer" \
--conf "spark.kryo.serializer.buffer.max=512m" \
--conf "spark.default.parallelism=48" \
--conf "spark.sql.shuffle.partitions=48" \
--conf "spark.memory.fraction=0.8" \
--conf "spark.memory.offHeap.enabled=true" \
--conf "spark.memory.offHeap.size=4g" \
--conf "spark.speculation=false" \
--conf "spark.locality.wait=0" \
--conf "spark.shuffle.compress=true" \
--conf "spark.shuffle.spill.compress=true" \
--conf "spark.shuffle.file.buffer=256k" \
--conf "spark.reducer.maxSizeInFlight=256m" \
--conf "spark.rdd.compress=true" \
--conf "spark.io.compression.codec=lz4" \
--conf "spark.sql.adaptive.enabled=false" \
--conf "spark.executor.memoryOverhead=2G" \
--conf "spark.executor.extraJavaOptions=-XX:+UseG1GC -XX
    :+UseStringDeduplication"
```

Cette configuration vise à garantir un compromis entre la stabilité, la reproductibilité et la performance du calcul distribué. Le choix de 8 exécuteurs dotés chacun de 4 cœurs et de 10 Go de mémoire permet de répartir efficacement les partitions sur le Cluster tout en évitant la saturation mémoire. L'utilisation du sérialiseur `Kryo` améliore la vitesse de transfert des objets entre les nœuds, réduisant ainsi la latence du `shuffle`.

Les paramètres de mémoire hors tas (`offHeap`) et la compression `LZ4` contribuent à limiter la pression sur le *garbage collector*, particulièrement sensible lors du traitement des grands graphes. Le choix du collecteur `G1GC` associé à l'option `UseStringDeduplication` favorise une gestion plus efficace des chaînes de caractères et une meilleure stabilité du temps d'exécution.

Enfin, la désactivation de l'exécution adaptative (`spark.sql.adaptive.enabled=false`) garantit la comparabilité entre les différentes structures de calcul (RDD et `DataFrame`) en évitant que Spark ne modifie dynamiquement le plan d'exécution au cours du *benchmark*.

Cette configuration influe directement sur les performances observées dans les résultats décrits dans ce rapport. Le paramètre `spark.default.parallelism=48` impose un degré de parallélisme constant entre les différentes exécutions, garantissant une répartition homogène des tâches sur l'ensemble des exécuteurs. Ce réglage favorise une utilisation efficace des ressources de calcul, tout en limitant les déséquilibres de charge susceptibles de fausser les comparaisons entre implémentations.

Les paramètres liés à la mémoire jouent également un rôle déterminant. L'activation de la mémoire hors tas (`offHeap`) et la définition d'une fraction mémoire élevée (`spark.memory.fraction=0.8`) permettent de réduire les accès disques et de mieux contrôler la pression exercée sur le *garbage collector*. Ces ajustements se traduisent par une diminution du temps consacré au nettoyage mémoire, notamment pour les grands graphes tels que `wiki-pt.txt` et `wiki-fr.txt`.

La désactivation de la spéculation (`spark.speculation=false`) empêche la relance de tâches supposées lentes, évitant ainsi les fluctuations de performance entre itérations successives. De même, la compression des échanges (`spark.shuffle.compress=true`) et la limitation de la taille des transferts inter-nœuds (`spark.reducer.maxSizeInFlight=256m`) réduisent le coût global des opérations de *shuffle*, particulièrement sollicitées lors des agrégations dans les implémentations RDD.

Enfin, le choix du collecteur `G1GC`, associé à la déduplication des chaînes de caractères, assure une gestion plus stable de la mémoire lors du calcul du PageRank. Cette configuration favorise une progression plus régulière du calcul itératif, en particulier pour les graphes de grande taille où les volumes de données échangées entre partitions sont significatifs.

De plus, le paramètre de stockage a été fixé à `MEMORY_AND_DISK_SER`, tandis que le facteur d'amortissement a été défini à 0,85.

La section suivante présente les résultats expérimentaux issus de ces exécutions, ainsi que les visualisations associées, permettant de comparer quantitativement le comportement de chaque approche.

VI. ANALYSE QUANTITATIVE DES TEMPS D'EXÉCUTION

La première phase d'analyse des performances de chaque implémentation s'est limitée à l'affichage successif du temps global associé à chaque graphe.

Le tableau 3 présente les temps d'exécution selon la taille des graphes et la méthode utilisée avec 10 itérations.

En supplément du tableau 3, la figure 3 regroupe deux visualisations complémentaires :

- la figure 3a montrant l'évolution des temps d'exécution en fonction de la taille des graphes ;
- la figure 3b comparant directement les quatre implémentations sur un même graphique synthétique.

Les résultats présentés dans la tableau 3 et illustrés par les figures 3a et 3b permettent d'évaluer l'influence de la structure de calcul (RDD ou `DataFrame`) et du partitionnement sur les performances globales de l'algorithme PageRank.

A. Comportement selon la taille des graphes

Pour les graphes de petite à moyenne taille (`wiki-cti.txt`, `wiki-breton.txt` et

TABLEAU 3
TEMPS D'EXÉCUTION DU PAGERANK SELON LA TAILLE DES GRAPHES ET LA MÉTHODE UTILISÉE (10 ITÉRATIONS).

Graphe	Nœuds	Arêtes	Méthode	Temps (s)
wiki-cthi.txt	15489	40124	RDD	75.21
			RDD Partitionné	59.88
			DF	103.25
			DF Partitionné	99.81
wiki-breton.txt	379763	1408847	RDD	242.17
			RDD Partitionné	223.82
			DF	176.15
			DF Partitionné	172.99
wiki-basque.txt	761282	4616203	RDD	338.33
			RDD Partitionné	379.41
			DF	284.74
			DF Partitionné	304.33
wiki-pt.txt	3765539	31571024	RDD	4067.32
			RDD Partitionné	801.45
			DF	597.40
			DF Partitionné	576.56
wiki-fr.txt	6740183	80920486	RDD	4831.34
			RDD Partitionné	2201.41
			DF	1348.86
			DF Partitionné	1884.48

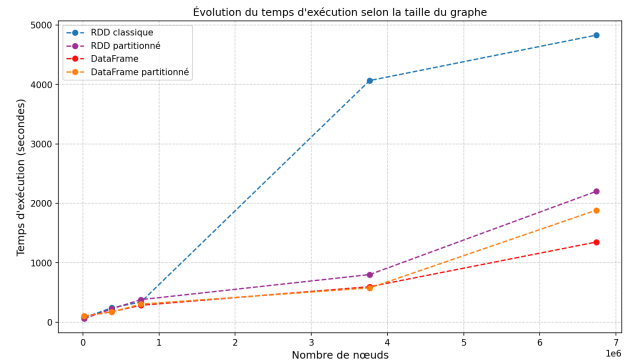
wiki-basque.txt), les temps d'exécution observés restent comparables entre les quatre implémentations. Les différences mesurées demeurent faibles, avec un avantage marginal pour la structure RDD classique. Cette tendance s'explique par la simplicité du modèle RDD, dont la planification et la sérialisation impliquent une surcharge moindre que celle introduite par le moteur DataFrame et son optimiseur Catalyst. Dans ces conditions, la granularité fine du partitionnement ne procure pas de bénéfice significatif, le coût des opérations restant dominé par la lecture et la distribution initiale des données.

B. Effet du partitionnement sur les grands graphes

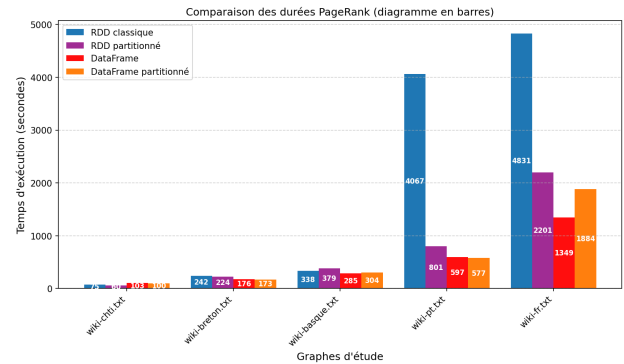
Lorsque la taille des graphes croît (wiki-pt.txt et wiki-fr.txt), les écarts de performance deviennent plus prononcés. L'implémentation RDD partitionné réduit considérablement les temps d'exécution par rapport à la version non partitionnée, avec des gains pouvant atteindre 75 %. Cette amélioration est directement liée à la réduction du nombre d'opérations de *shuffle* et à une meilleure localité des données grâce à l'utilisation d'un *HashPartitioner* cohérent entre les transformations successives. Toutefois, malgré cette optimisation, le modèle RDD partitionné demeure moins performant que les implémentations basées sur DataFrame, qui bénéficient d'un plan d'exécution optimisé et d'une gestion mémoire plus efficace.

C. Limites du partitionnement manuel dans les DataFrames

L'analyse met en évidence un comportement inattendu : la version DataFrame Partitionné présente des performances légèrement inférieures à la version DataFrame



(a) Évolution des temps d'exécution selon la taille des graphes.



(b) Comparaison directe des temps d'exécution par implémentation (diagramme en barres).

FIGURE 3. Comparaison des temps d'exécution du PageRank selon la taille des graphes et la structure de calcul utilisée.

standard. Ce résultat suggère que le partitionnement manuel ne se substitue pas efficacement à la répartition automatique opérée par Spark. Les mécanismes internes d'optimisation, notamment le plan d'exécution adaptatif et la planification dynamique de Catalyst, permettent une répartition plus équilibrée des partitions entre les *exécuteurs*. L'imposition d'un partitionnement fixe tend, au contraire, à restreindre la flexibilité du moteur et à accroître les échanges inter-partitions, ce qui explique la légère dégradation constatée.

D. Synthèse des résultats

L'étude des temps d'exécution permet de dégager plusieurs constats :

- pour les graphes de petite et moyenne taille, les quatre approches présentent des performances similaires, avec une légère supériorité du modèle RDD ;
- sur les graphes volumineux, le partitionnement améliore les performances du modèle RDD, mais ce dernier reste inférieur à la version DataFrame en termes de rapidité ;
- le partitionnement manuel des DataFrames ne procure pas de bénéfice et tend à perturber les optimisations automatiques du moteur Spark.

Ces observations mettent en évidence le rôle central de la stratégie de partitionnement dans les traitements distribués à grande échelle. Elles confirment également que les optimisations internes des DataFrames surpassent, dans la majorité des cas, les réglages manuels effectués sur les structures RDD, en particulier lorsque la volumétrie du graphe devient importante.

Abordons à présent une étude plus précise des temps d'exécution sur la base des quatre opérations : `map`, `flatMap`, `sum` et `count`.

VII. ANALYSE DES TEMPS D'EXÉCUTION PAR OPÉRATION

Sur la base des résultats précédents, il semble pertinent d'étudier plus finement les temps d'exécution selon les quatre opérations décrites ci-après et qui reflètent directement les coûts associés aux transformations et aux actions appliquées aux données distribuées :

- **map** : applique une fonction à chaque élément du RDD ou du DataFrame. Cette opération est purement locale et s'exécute indépendamment sur chaque partition. L'analyse de son temps d'exécution permet d'évaluer l'efficacité du traitement élémentaire et la gestion interne de la parallélisation.
- **flatMap** : étend la transformation précédente en produisant plusieurs éléments de sortie pour chaque élément d'entrée. Elle intervient notamment dans la construction du graphe et la propagation des contributions lors du calcul du PageRank. Son coût est plus sensible à la structure des données et au volume intermédiaire généré, ce qui en fait un bon indicateur de la surcharge de *sérialisation* et des transferts réseau.
- **sum** : agrège l'ensemble des valeurs numériques distribuées au sein du Cluster. Cette opération déclenche un processus de réduction global impliquant des échanges inter-partitions. L'analyse de ses performances renseigne sur l'efficacité des agrégations et sur la latence des communications entre *executors*.
- **count** : évalue le nombre total d'éléments dans la structure de données. A première vue simple, cette action force l'exécution complète du plan de calcul et permet ainsi de mesurer le coût global de matérialisation des transformations précédentes. Elle constitue un indicateur pertinent de la charge d'exécution effective et du coût des opérations de lecture et de synchronisation.

L'étude de ces quatre opérations offre une vision complémentaire aux mesures globales du tableau tableau 3. Elle permet d'isoler les facteurs qui influencent le plus directement les performances, tels que la localité des calculs, la *sérialisation* des objets, le volume de données échangé ou encore la stratégie d'évaluation paresseuse (*lazy evaluation*) propre à Spark.

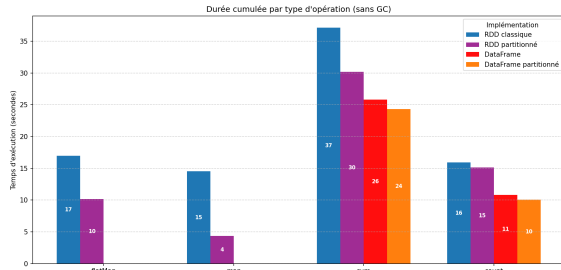
Les figures 4a à 4j présentent la durée des principales opérations (`map`, `flatMap`, `sum`, `count`) pour chaque structure de calcul et chaque graphe, avec et sans prise

en compte du temps consacré au *garbage collector*. Les graphiques de gauche illustrent les temps d'exécution bruts, tandis que ceux de droite intègrent le surcoût lié à la gestion mémoire.

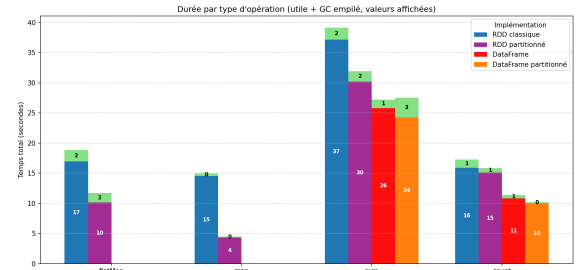
L'analyse met en évidence que l'implémentation DataFrame ne mobilisant pas les opérations `map` et `flatMap`, à la différence de l'approche en RDD, constitue un premier facteur explicatif de la meilleure efficacité des DataFrames, en limitant le nombre de transformations explicites sur les partitions.

Concernant les opérations `sum` et `count`, bien que les temps globaux soient supérieurs pour les petits graphes avec la structure DataFrame (voir figure 3), ces opérations sont exécutées plus rapidement que dans les RDDs. Ce comportement s'explique par la surcharge initiale du moteur Catalyst, responsable de la génération et de l'optimisation du plan d'exécution. Sur de petits volumes de données, cette phase d'analyse et de planification introduit un coût fixe significatif, masquant les gains offerts par les optimisations internes. Lorsque la taille du graphe croît, ce coût devient marginal et les optimisations de Catalyst se traduisent par une réduction sensible du temps total d'exécution.

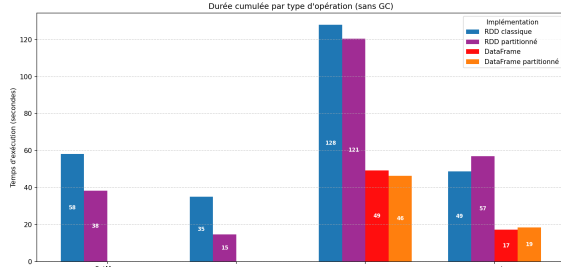
Enfin, le temps induit par le *garbage collector* croît proportionnellement à la taille des graphes et au volume de mémoire utilisé. Pour les graphes les plus volumineux, tels que `wiki-pt.txt` et `wiki-fr.txt`, ce coût devient particulièrement marqué dans l'implémentation RDD sans partitionnement, où la gestion mémoire génère une surcharge importante. Cette contrainte explique la baisse notable de performance observée pour les graphes denses, en raison de l'augmentation de la fréquence et de la durée des cycles de nettoyage.



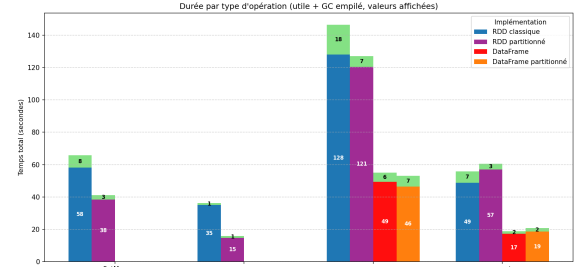
(a) wiki-cthi — sans GC



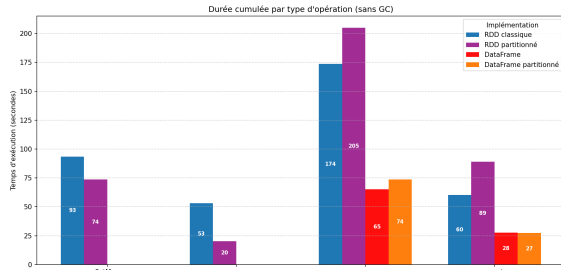
(b) wiki-cthi — avec GC



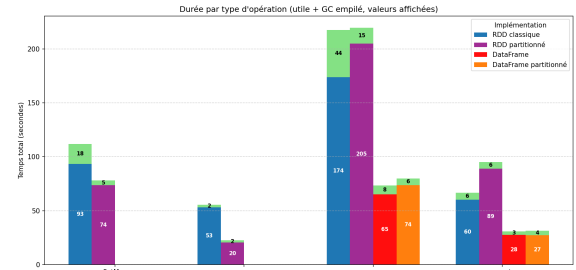
(c) wiki-breton — sans GC



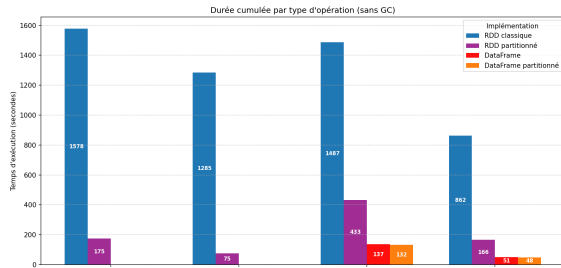
(d) wiki-breton — avec GC



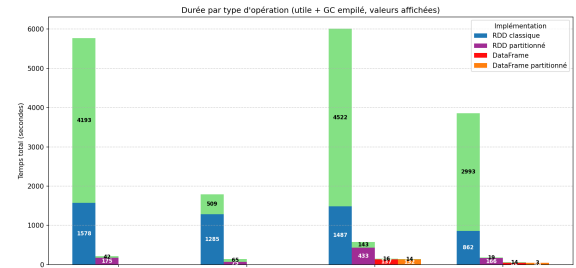
(e) wiki-basque — sans GC



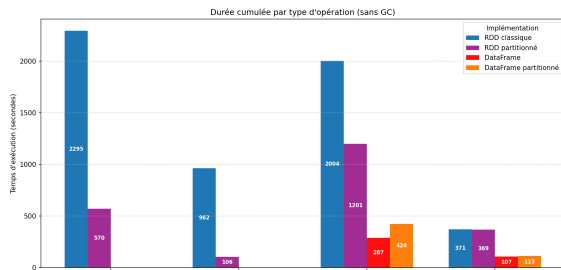
(f) wiki-basque — avec GC



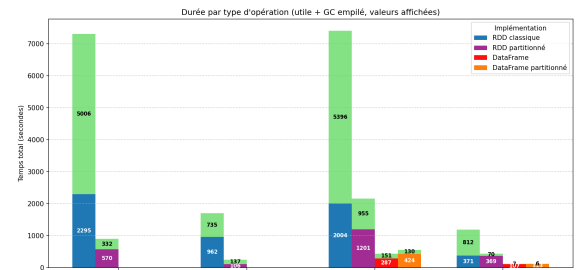
(g) wiki-pt — sans GC



(h) wiki-pt — avec GC



(i) wiki-fr — sans GC



(j) wiki-fr — avec GC

FIGURE 4. Durée moyenne par type d'opération (map, flatMap, sum, count) pour chaque structure de calcul et chaque graphe, avec et sans prise en compte du temps passé dans le *garbage collector*.

L'étude présentée a permis d'analyser en profondeur l'implémentation de l'algorithme PageRank sur la plateforme Spark, en comparant quatre structures de calcul : RDD, RDD partitionné, DataFrame et DataFrame partitionné. L'ensemble des expérimentations a été mené sur cinq graphes de tailles croissantes, à l'aide d'une configuration Spark optimisée, afin d'évaluer l'impact des choix d'implémentation sur les temps d'exécution et l'efficacité mémoire.

Les résultats ont mis en évidence plusieurs constats majeurs. Tout d'abord, pour les graphes de petite et moyenne taille, les structures RDD offrent des performances comparables, voire légèrement supérieures, à celles des DataFrames. Cependant, à mesure que la taille du graphe augmente, les DataFrames tirent parti du moteur d'optimisation Catalyst, qui améliore la planification et la réduction des échanges réseau, leur conférant une meilleure scalabilité.

L'introduction d'un partitionnement explicite s'est révélée bénéfique pour les RDDs, réduisant de manière significative les coûts liés aux *shuffles* et à la communication inter-nœuds. Cette optimisation permet d'obtenir un gain de performance notable sur les graphes volumineux, bien que l'implémentation reste moins efficace que celle basée sur les DataFrames. À l'inverse, le partitionnement manuel des DataFrames n'a pas apporté d'amélioration et a même diminué les performances, confirmant que le moteur Spark gère déjà efficacement la distribution et l'adaptation du plan d'exécution à la taille du jeu de données.

L'analyse des durées d'exécution par type d'opération (`map`, `flatMap`, `sum`, `count`) a montré que les DataFrames réduisent significativement le coût de ces opérations grâce à une exécution vectorisée et une meilleure gestion mémoire. De plus, l'étude du *garbage collector* a souligné son impact croissant sur les performances, notamment pour les graphes de grande taille, où il devient un facteur limitant dans le cas des RDDs non partitionnés.

En conclusion, il ressort de cette analyse que :

- le partitionnement constitue une amélioration essentielle pour les RDDs, leur permettant d'approcher les performances des DataFrames ;
- les DataFrames conservent un avantage global en termes de scalabilité et d'optimisation automatique ;
- la surcharge induite par le moteur Catalyst devient négligeable pour les graphes de grande taille, mais pénalise les petits jeux de données ;
- la gestion de la mémoire et du *garbage collector* représente un axe critique d'optimisation pour les traitements distribués de grande échelle.

Ces observations ouvrent la voie à des perspectives d'amélioration, notamment par l'intégration de mécanismes de partitionnement adaptatif, l'étude de formats de stockage optimisés (par exemple parquet) ou encore l'analyse de l'impact du paramétrage du *garbage collector* sur les performances globales. De telles optimisations permettraient

GLOSSAIRE

Dangling node Nœud sans lien sortant dans le graphe. Dans l'algorithme PageRank, ces nœuds redistribuent leur poids de manière uniforme sur l'ensemble du graphe. . 2, 4–8

API Une API, ou interface de programmation d'application, est un ensemble de définitions et de protocoles qui facilite la création et l'intégration des applications. Les API permettent aux diverses applications d'un système de communiquer sans connaître les détails de leur mise en œuvre. Simplifiant ainsi leur développement . 1, 7, 8

Catalyst Moteur d'optimisation de requêtes de Spark, intégré au module *Spark SQL*. Il transforme les opérations déclaratives exprimées sur les *DataFrame* en plans d'exécution physiques optimisés, en appliquant des règles de réécriture et des optimisations logiques et physiques. . 1, 4, 6–8, 10, 11, 13, 14

Chaîne de Markov Modèle mathématique décrivant un système dynamique où la probabilité de passer d'un état à un autre dépend uniquement de l'état courant, indépendamment des états précédents. Utilisé pour modéliser le comportement de navigation d'un utilisateur sur le web. . 14

Cluster Ensemble de machines physiques ou virtuelles interconnectées, utilisées pour exécuter des calculs distribués. Dans le cadre de Spark, un cluster est composé d'un *driver* coordonnant l'exécution et de plusieurs *executors* responsables du traitement parallèle des partitions de données. . 3, 5–9, 11, 14

DataFrame Abstraction de plus haut niveau dans Spark, analogue à une table relationnelle. Les *DataFrames* permettent d'exprimer les opérations de transformation sous forme déclarative, optimisée par le moteur Catalyst, tout en assurant la distribution des calculs sur le Cluster. . 1, 3, 4, 6–11, 13, 14

Facteur d'amortissement Coefficient (généralement noté α ou d) représentant la probabilité qu'un internaute continue de suivre un lien hypertexte lors du calcul du PageRank. Sa valeur typique est 0,85. . 2

Matrice de transition Représentation matricielle d'une Chaîne de Markov, dont chaque coefficient indique la probabilité de transition entre deux états du système. . 2

PageRank Algorithme de classement des pages proposé par Larry Page et Sergey Brin, fondé sur la structure

des hyperliens du web. Il attribue à chaque page un score reflétant son importance relative. . 1–6, 8–11, 13, 14

parquet Apache Parquet est un format de fichier de données open source en colonnes, conçu pour stocker et récupérer des données avec une grande efficacité. Il fournit d'excellents schémas de compression et d'encodage des données. Ses performances supérieures lui permettent de traiter des données complexes en vrac. Apache Parquet a pour but d'être un format d'échange commun pour les charges de travail tant interactives que par *batch* . 13

Resilient Distributed Dataset (RDD) Structure de données fondamentale de Spark, représentant un ensemble distribué et tolérant aux pannes d'éléments partitionnés entre les nœuds du Cluster. Les RDD permettent des transformations parallisées et immuables, offrant un contrôle fin sur le partitionnement et la persistance des données. . 1, 3–11, 13

Sites fortement connectés (SFC) Ensemble de pages web formant une sous-structure densément interconnectée, représentant le cœur du graphe du web. . 2

Spark Framework de calcul distribué open source conçu pour le traitement rapide et tolérant aux pannes de larges volumes de données. Il fournit plusieurs API de haut niveau (RDD, *DataFrame*, *Dataset*) et un moteur d'exécution optimisé pour les applications itératives telles que le calcul du PageRank. . 1, 3, 4, 9–11, 13, 14

stochastique Une matrice est dite stochastique lorsque la somme de chaque colonne vaut 1 . 2

BIBLIOGRAPHIE

- [1] L. PAGE et S. BRIN, « The Anatomy of a Large-Scale Hypertextual Web Search Engine, » in *Proceedings of the Seventh International World Wide Web Conference*, 1998. adresse : <http://infolab.stanford.edu/pub/papers/google.pdf>.
- [2] L. PAGE, « Method for node ranking in a linked database, » US 6,285,999 B1, Submitted on Jan. 9, 1998, 2001.